SI 100B Programming Assignment 1

SI 100B TA Team

September 26, 2025

Notice

- This homework is due **21:00 October 9, Thursday**, please start early.
- The problems should be solved **individually**. You should submit solutions for all problems to the **online judge** (OJ) system, which will give a score for each submission automatically.
- You may discuss course material, programming languages, general techniques, and share test cases with others.
- You're encouraged to ask any question about problem description, environment setup, error information, grammar, functions, methods etc., or post general ideas at Piazza. It can help others too!
- However, you may **NOT** read, possess, copy, or submit any part of other's solution. You may not ask others to directly debug your program or provide specific lines of solution. You should not access another person's account. Also, do not offer such help, and protect your account and code. Giving or receiving inappropriate help is **plagiarism**, an infringement of academic integrity.
- You are not allowed to write your solution directly by or based on code generated by **AI tools**.
- **Good luck and have fun!**

1 Lower-Triangular Multiplication Table (20 points)

1.1 Task

Given a positive integer `n` ($1 \leq n \leq 9$) and print a n-row **lower-triangular** multiplication table:

- Row $i (1 \leq i \leq n)$ contains i expressions: $j \times i =$ product (j runs from 1 to i)
- Expressions are concatenated **with a tab character** `\t`;
- The product of the multiplication must be formatted to occupy a fixed width of two character spaces. If the product is a single-digit number (e.g. 7), a space should be prepended to it (e.g. ” 7”). If the product is a two-digit number (e.g., 12), it will naturally fill the entire two-character space.
- You need to consider the validity of the input. If the program receives an invalid input, please output ”Invalid”!

1.2 Input/Output Specification

1.2.1 Input

A positive integer `n`.

1.2.2 Output

A lower triangular multiplication table with n rows.

1.2.3 Example

1. Input

```
4
```

   Output

```
1x1= 1
1x2= 2  2x2= 4
1x3= 3  2x3= 6  3x3= 9
1x4= 4  2x4= 8  3x4=12  4x4=16
```

2. Input

```
12
```

Output

```
Invalid
```

3. Input

```
2.1
```

Output

```
Invalid
```

1.3 Hint

- Use `f'{j}×{i}={i*j:2}'` to fix the width of each expression;
- If a floating-point variable `n` is an integer, then the values of `int(n)` and `n` are equal.
- The OJ (Online Judge) system will ignore trailing spaces and tabs at the end of each line in your output text.

2 Linear Interpolation (20 points)

Linear interpolation is a common technique in graphic drawing. Given two points (x_1, y_1) and (x_2, y_2), along with an intermediate value x, you can calculate the corresponding y value.

$$y = y_1 + \frac{y_2 - y_1}{x_2 - x_1} * (x - x_1)$$

2.1 Task

Input five floating-point numbers: x_1, y_1, x_2, y_2, x, Suppose $x_1 \neq x_2$. Use the linear interpolation formula to calculate and output the corresponding y value, retaining two decimal places.

2.2 Input/Output Specification

2.2.1 Input

The parameters x_1, y_1, x_2, y_2, x, one line for each.

2.2.2 Output

y retaining two decimal places.

2.2.3 Example

1. Input

```
5
4
6
7
15
```

Output

```
34.00
```

2.3 Hint

- You might need to consider how to receive so many values in different rows.
- There are a few ways to format floating point numbers to a fixed number of decimal. We encourage you to explore different methods.

  Examples of formatting floating point numbers:

```
# Method 1: Using .format()
'{:.1f}'.format(1.19)  # Output: '1.2'

# Method 2: Using f-strings
f'{1.19:.1f}'          # Output: '1.2'
```

3 Rhombus Generator (20 points)

David is a five-year-old child. He especially likes to build jigsaw puzzles with rhombus blocks. But his dog, named Frod, always hid his Rhombus blocks. Please complete a rhombus generator to help David!

3.1 Task

Your program must generate a rhombus pattern from the terminal using the given characters. David will tell you the height and the difference in the number of patterns between every two adjacent rows of the rhombus, which are presented by h and w, as well as the string David specified, represented by c. Design your program to complete this small task!

3.2 Input/Output Specification

3.2.1 Input

The parameters h, w, c, one line for each. h is odd number and w is even number.

3.2.2 Output

A rhombus pattern.

3.2.3 Example

1. Input

```
5
2
-
```

   Output

```
  -
 ---
-----
 ---
  -
```

2. Input

```
3
4
o
```

Output

```
  o
ooooo
  o
```

4 Square-Root Approximation (20 points)

Ancient Greeks had no square-root button; they used a special way to calculate $\sqrt{a}$. You can do the same on your keyboard—guess once, then keep averaging the guess with a divided by the guess. This method converges very quickly.

4.1 Task

Read a positive real a and approximate $\sqrt{a}$ by the **bisection-like iteration**:

- Initialize: guess $= \frac{a}{2}$
- Update: guess $= \frac{\text{guess} + \frac{a}{\text{guess}}}{2}$

After **20 iterations** print the result with **20 decimal places**.

Note: Make sure to use the method mentioned above. Other methods for calculating square-root, such as using `math` module, will fail to pass some test cases.

4.2 Input/Output Specification

4.2.1 Input

A positive real a.

4.2.2 Output

The 20th iterate, 20 decimal places

4.2.3 Example

1. Input

```
2
```

Output

```
1.41421356237309492343
```

4.3 Hint

- Use a simple loop for 20 updates;
- Only two variables are required: $guess$ and a.

5 Palindromic Substring Counter (20 points)

A palindromic string reads identically from left to right and right to left (e.g. `level`, `aa`, or single letter `A`). Here you must count **every continuous substring** that is palindromic; substrings with the same content but different starting positions are considered different. Refer to **Hint** for better understanding.

5.1 Task

Given a string `s`, output the total number of its palindromic substrings.

- Length: $1 \leq |s| \leq 2000$
- Characters: letters only (case-sensitive)

5.2 Input/Output Specification

5.2.1 Input

One line, the string `s`.

5.2.2 Output

The total count of palindromic substrings (A positive integer).

5.2.3 Example

1. Input

```
abbaeae
```

   Output

```
11
```

2. Input

```
aaa
```

   Output

```
6
```

5.3 Hint

Centre-expansion method: **Treat each character or the gap between adjacent characters as a potential center of a palindrome.** Expand outward symmetrically (left and right); whenever matching characters are found, a new palindromic substring is identified—continue expanding further. Scan all possible centers once and accumulate the total count.

Here is an example for the input `abbaeae`.

Part 1: Finding Odd-Length Palindromes

Index	Palindrome(s) Found	Running count
s[0] (a)	a	1
s[1] (b)	b	2
s[2] (b)	b	3
s[3] (a)	a	4
s[4] (e)	e, then aea	6
s[5] (a)	a, then eae	8
s[6] (e)	e	9

Table 1: Palindrome(s) Found and Running Count

First, we treat a as a potential centre. According to the center-expansion method, we can find 1 palindromic substring (i.e., a itself).

Then, following this rule, we proceed to iterate until the 5th character "e". In this iteration, using the center-expansion method, we discover 2 palindromic substrings (e and aea). As a result, after this iteration, the total count of palindromic substrings increases from 4 to 6.

By continuing this iteration until the end of the sequence, we can accurately count the total number of palindromic substrings.

Part 2: Finding Even-Length Palindromes

Under this setup, we need to treat the gaps between characters as potential centers, with the specific iteration process being similar to Table 1.

Index	Palindrome(s) Found	Running count
s[0]–s[1] (a–b)	(None, $a \neq b$)	9
s[1]–s[2] (b–b)	bb, then abba	11
s[2]–s[3] (b–a)	(None, $b \neq a$)	11
s[3]–s[4] (a–e)	(None, $a \neq e$)	11
s[4]–s[5] (e–a)	(None, $e \neq a$)	11
s[5]–s[6] (a–e)	(None, $a \neq e$)	11

Table 2: Palindrome detection for adjacent character pairs

Final Total: `count = 11`